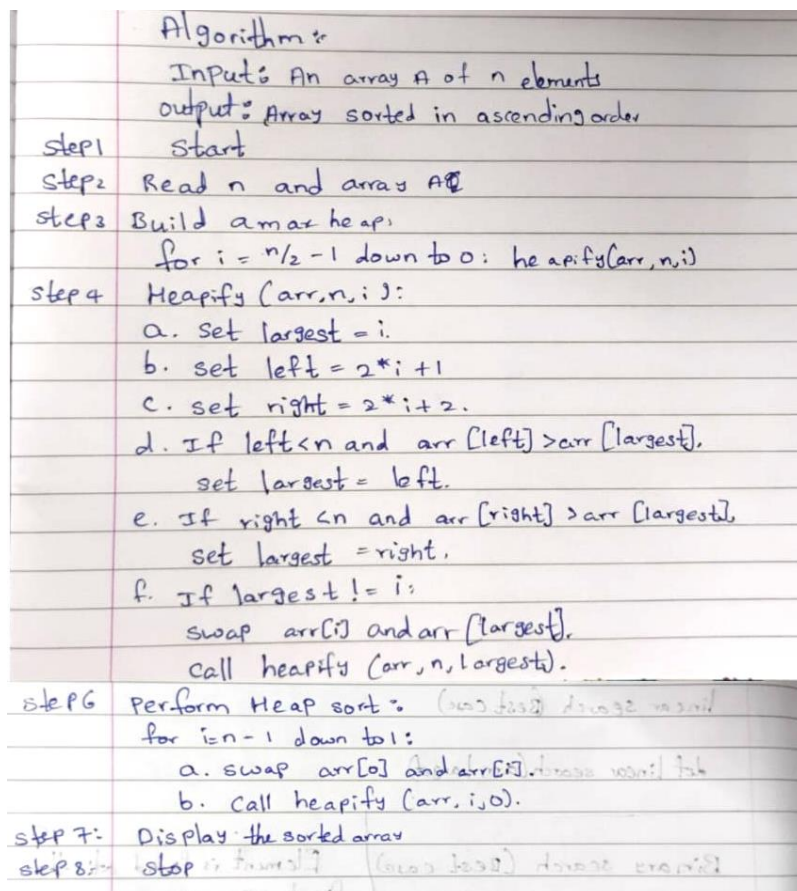


PRACTICAL - 3

AIM :Implementation of max-heap sort algorithm

ALGORITHM :



PROGRAM :

```
#include <iostream>
```

```
using namespace std;
```

```
void heapify(int arr[], int n, int i)
```

```
{
```

```
    int largest = i;
```

```
    int left = 2 * i + 1;
```

```
    int right = 2 * i + 2;
```

```
    if (left < n && arr[left] > arr[largest])
```

```
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i)
    {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n)
{
    // Build Max Heap
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Heap Sort
    for (int i = n - 1; i > 0; i--)
    {
        swap(arr[0], arr[i]);
        heapify(arr, i, 0);
    }
}

int main()
{
    int n;
    int arr[100];

    cout << "Enter size of array: ";
    cin >> n;

    cout << "Enter array elements: ";
    for (int i = 0; i < n; i++)
        cin >> arr[i];

    heapSort(arr, n);

    cout << "Sorted array: ";
```

```
for (int i = 0; i < n; i++)  
    cout << arr[i] << " ";  
  
return 0;  
}
```

Output

```
Enter size of array: 5  
Enter array elements: 23  
45  
12  
2  
3  
Sorted array: 2 3 12 23 45
```

Time complexity
Best case :



heap sort best case

for ($i = n/2 - 1$; $i \geq 0$; $i--$) $-O(n)$

 heapify (arr, n, i); $-O(n)$

for ($i = n - 1$; $i > 0$; $i--$) $-n$

{

 swap(arr[0], arr[i]); $-n$

 heapify (arr, i, 0); $O(\log n)$

}

$T(n) = O(n) + O(n) + O(n) + O(n) + O(\log n)$

$= O(n \log n)$

Average case :

for ($i = n/2 - 1$; $i \geq 0$; $i--$) $-O(n)$

 heapify (arr, n, i); $-O(n)$

for ($i = n - 1$; $i > 0$; $i--$) $-n$

{

 swap(arr[0], arr[i]); $-n$

 heapify (arr, i, 0); $O(\log n)$

}

$T(n) = O(n) + O(n) + O(n) + O(n) + O(\log n)$

$= O(n \log n)$



Worst case :

for ($i = n/2 - 1$; $i \geq 0$; $i--$) $- O(n)$
 heapify (arr, n, i); $- O(n)$
for ($i = n - 1$; $i > 0$; $i--$) $- n$
{
 swap(arr[0], arr[i]); $- n$
 heapify (arr, i, 0); $O(\log n)$
}
 $T(n) = O(n) + O(n) + O(n) + O(n) + O(\log n)$
 $= O(n \log n)$